

HBTXR: Algorithm–Hardware Co-Designed Hybrid Eye Tracking for On-Device Extended Reality

Anonymous Author(s)

Abstract—Real-time eye tracking on extended reality (XR) devices must satisfy tight latency, energy, memory, and form-factor constraints while preserving robust pupil localization and gaze estimation. Prior work has often optimized either algorithms or accelerators in isolation, which leads to mismatches between model behavior and deployment cost. We present HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework that unifies frame-guided Search, event-guided Track, runtime scheduling, and accelerator mapping in one system pipeline. At the algorithm level, HBTXR introduces a time-synchronous and geometry-stable hybrid supervision flow, a modality-aware transformer, and an implicit model pruning flow that distills a deployment student in which Search executes the full backbone while Track exits from a front-half cut point. A runtime-aware scheduler then governs mode switching from quality signals. At the hardware level, HBTXR adopts quantization-aware operator reformulation, table-driven non-linear approximation, and a streaming multi-stage pipeline that jointly optimizes intra-layer dataflow and cross-layer coupling. Experimental results show that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68° , and an end-to-end Track-mode latency of 0.57 ms, while maintaining 97.8% Search success rate and 1754 Hz effective update rate in Track mode. These results suggest that HBTXR offers a practical system point for accurate, low-latency, and deployment-oriented XR eye tracking.

Index Terms—Eye tracking, extended reality, hybrid sensing, event camera, transformer accelerator, FPGA, algorithm–hardware co-design.

I. INTRODUCTION

Eye tracking has become a core enabling function for extended reality (XR), supporting gaze-based interaction, foveated rendering, user-state understanding, and privacy-preserving on-device perception [1], [2]. In practice, however, XR eye tracking must satisfy three requirements simultaneously: it must remain semantically robust under appearance changes, temporally responsive under rapid eye motion, and efficient enough to run on embedded hardware with strict power and memory budgets. This combination makes XR eye tracking fundamentally a system-level design problem rather than a stand-alone perception problem.

Existing approaches reveal a structural trade-off. Frame-based methods are strong in semantic recovery and calibration-aware regression, but repeated dense-image processing introduces high bandwidth and latency cost [3]. Event-based methods offer sparse sensing and high temporal resolution, but their reliability degrades under weak-event regimes, drift accumulation, or reinitialization failures [4]–[6]. Hybrid methods improve this balance, yet many still treat fusion, runtime mode switching, and hardware deployment as loosely coupled stages [7].

This paper addresses that gap through HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system for on-device XR. In contrast to monolithic multimodal regression, HBTXR explicitly separates *Search* and *Track*. Search is frame-guided and refresh-oriented, designed for robust relocalization and anchor recovery. Track is event-guided and state-persistent, designed for low-latency residual update around the current pupil estimate. This decomposition is reflected not only in the model architecture, but also in the supervision flow, training pipeline, runtime scheduler, and hardware architecture. In the full model, both modalities are learned under one shared transformer family, and the deployment-oriented implicit model pruning stage distills a slim student in which Search preserves the full backbone while Track exits from a front-half cut point and forwards the intermediate representation directly to the Track head.

Fig. 1 summarizes the overall design. HBTXR first constructs time-synchronous supervision so that frame and event inputs share one stable ellipse-centric target. It then employs a modality-aware transformer with a shared backbone and decoupled heads, followed by an implicit model pruning step that exports a smaller deployment student with a Track-side cut point. A runtime scheduler switches between Search and Track according to confidence, similarity, event density, and eye-state validity. Finally, a CPU–FPGA deployment architecture maps the shared backbone and lightweight heads to differentiated execution paths. In this way, HBTXR internalizes hardware constraints during algorithm design and tunes the hardware around the actual runtime semantics of Search and Track [12].

The key contributions of this work are summarized as follows.

- We reformulate hybrid eye tracking as a Search–Track problem and introduce a time-synchronous, geometry-stable supervision contract for aligning frame, event, and state targets.
- We design a modality-aware transformer with shared backbone and decoupled Search, Event, Track, and Mask heads, and couple it with an implicit model pruning flow that distills a deployment student where Search uses the full backbone and Track exits from a front-half cut point.
- We develop a runtime-aware scheduler that explicitly manages Search mode, Track mode, and mode switching according to quality assessment signals.
- We present a deployment-oriented accelerator design with quantized operators, table-driven non-linear approximation, streaming execution, multi-stage pipeline balance, and CPU–FPGA co-scheduling.

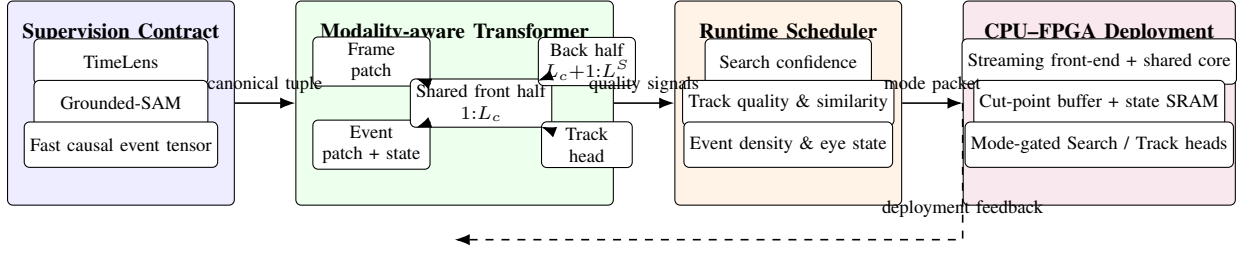


Fig. 1. Overview of HBTXR. The framework combines a time-synchronous supervision contract, a modality-aware Search/Track transformer, a runtime scheduler, and a deployment-oriented CPU-FPGA pipeline. Search traverses the full student backbone, whereas Track exits from the front-half cut point introduced by implicit model pruning.

II. RELATED WORKS

A. Frame-based Eye Tracking

Frame-based eye tracking remains the dominant paradigm in near-eye and head-mounted systems because grayscale or infrared frames preserve strong semantic structure for pupil localization, iris fitting, and gaze regression [1], [3]. These methods are particularly effective for global relocalization and calibration-aware estimation. However, their strengths come with dense sensing and repeated full-frame processing, which translate into high memory traffic, bandwidth overhead, and limited responsiveness under rapid eye motion.

B. Event-based Eye Tracking

Event-based eye tracking exploits asynchronous brightness-change sensing to achieve sparse and high-temporal-resolution processing. Prior work has shown that event-driven methods are attractive for low-latency update during fast saccades and for reducing redundant processing during fixation [4]–[6]. Nonetheless, event-only pipelines remain weaker in semantic stabilization, blink recovery, and long-horizon relocalization, especially when event density becomes low or the tracker drifts away from the valid pupil contour.

C. Hybrid Frame-Event Eye Tracking

Hybrid eye tracking combines frame-based semantic recovery with event-based temporal responsiveness. Representative systems demonstrate that relocalization can be executed on low-rate frames while event streams support high-rate update [7]. This direction is promising for XR because it aligns sensing modality with tracking phase. However, many hybrid systems still leave the relationship between fusion strategy, runtime mode policy, and hardware mapping under-specified.

D. Deployment-Oriented and Hardware Accelerator

Deployment-oriented eye tracking has been studied through frame-centric accelerators, event-centric sparse designs, and ASIC-style low-power trackers [8]–[10]. These works confirm that eye tracking is not only a perception problem but also a hardware-design problem. In parallel, algorithm–hardware co-optimization has been shown to be effective in edge vision systems by integrating model design, quantization, and accelerator mapping in one workflow [12]. The remaining gap for hybrid XR eye tracking is a unified framework that co-designs Search/Track semantics, transformer inference, and deployment behavior together.

E. Summary

The literature suggests four conclusions. First, frame-centric methods remain the most reliable choice for global refresh. Second, event-centric methods remain the best candidate for ultra-low-latency local update. Third, hybrid systems are compelling only when their operating modes are made explicit. Fourth, deployment efficiency improves when hardware constraints are internalized during model design. HBTXR is built precisely around these conclusions.

III. PROPOSED HBTXR ALGORITHM

A. Framework Design

1) *Problem Formulation*: HBTXR operates on a hybrid sensing input composed of a frame stream, an event stream, and a recurrent state. Let $I_t \in \mathbb{R}^{H \times W}$ denote the frame captured at time t , and let $E_t = \{e_k = (x_k, y_k, p_k, \tau_k)\}_{k=1}^{N_t}$ denote the event set within the current window, where (x_k, y_k) is the event coordinate, $p_k \in \{-1, +1\}$ is the polarity, and τ_k is the timestamp. The event stream is transformed into a polarity-split tensor $V_t \in \mathbb{R}^{2 \times H \times W}$ through a causal accumulation operator.

Eye Model: The target pupil state is parameterized as

$$\mathbf{s}_t = [x_t, y_t, a_t, b_t, u_t, v_t] \in \mathbb{R}^6, \quad (1)$$

where (x_t, y_t) is the pupil center, (a_t, b_t) are the ellipse axes, and $[u_t, v_t] = [\sin(2\theta_t), \cos(2\theta_t)]$ is the trigonometric rotation encoding. This representation avoids discontinuity around the angular wrap-around boundary and supports stable regression of ellipse orientation.

Objectness: Rather than only predicting ellipse parameters, HBTXR also models *objectness*, namely the likelihood that the current observation contains a valid and trackable pupil hypothesis. We denote the binary training target by

$$o_t^* = \mathbf{1}[\text{valid visible pupil at time } t]. \quad (2)$$

In Search mode, objectness is interpreted as refresh confidence that indicates whether a frame-guided candidate should become the new anchor. In Track mode, objectness is paired with track confidence and track quality to assess whether the current event-driven residual update is still reliable. We reserve z_t for the separate closed-eye or invalid-observation flag used by the scheduler, so that objectness and eye-state validity are not overloaded by the same symbol. This objectness formulation becomes the interface between perception and scheduling.

2) *Framework Overview*: The overall framework consists of four interacting components: a supervision pipeline, a modality-aware transformer, a Search/Track scheduler, and a deployment path. Search performs frame-guided relocalization and anchor refresh. Track performs event-guided residual update conditioned on the previous state. During deployment-oriented implicit model pruning, the Search path keeps the full backbone while the Track path is bound to a front-half cut point so that the slim student can early-exit without changing the scheduler-visible outputs. This explicit separation allows HBTXR to treat hybrid eye tracking as a system with two operating modes rather than as one undifferentiated multimodal regressor. The deployed estimate is therefore written as

$$\hat{\mathbf{s}}_t = \begin{cases} \hat{\mathbf{s}}_t^s, & m_t = \text{Search}, \\ \bar{\mathbf{s}}_{t-1} \oplus \Delta \hat{\mathbf{s}}_t, & m_t = \text{Track}, \end{cases} \quad (3)$$

where $\oplus$ denotes the residual ellipse-state update operator.

B. Time-Synchronous and Geometry-Stable Hybrid Supervision

Dataset Problem: A major difficulty in hybrid eye tracking is that frame labels are usually sparse in time whereas event data are asynchronous and dense. If supervision timestamps are not aligned carefully, the model is forced to learn inconsistent geometry across frame, event, and recurrent inputs. HBTXR addresses this issue by constructing one canonical supervision contract in which frame cues, event cues, previous state, masks, and quality metadata all refer to the same time index and the same ellipse parameterization.

1) *Frame Interpolation (TimeLens)*: To densify supervision in time, HBTXR first uses frame interpolation to synthesize intermediate frame observations between sparsely labeled frames. We describe this stage as a TimeLens-style interpolation block, whose purpose is not to create a new inference modality at runtime, but to construct better-aligned supervisory frames offline. The interpolated frames reduce the temporal gap between event windows and annotation anchors, which is particularly important during rapid saccades.

2) *Dense Annotation (Grounded-SAM)*: After temporal densification, HBTXR applies a Grounded-SAM-assisted pseudo-labeling stage that refines eye-region and pupil masks from the interpolated frames. Let $\bar{M}_t$ denote the pupil mask and $\bar{B}_t$ denote the eye-region target at the aligned time index. The ellipse target is recovered from the mask moments as

$$\bar{\mathbf{c}}_t = \frac{1}{|\bar{M}_t|} \sum_{(x,y) \in \bar{M}_t} [x, y]^\top, \quad (4)$$

$$\bar{\Sigma}_t = \frac{1}{|\bar{M}_t|} \sum_{(x,y) \in \bar{M}_t} ([x, y]^\top - \bar{\mathbf{c}}_t)([x, y]^\top - \bar{\mathbf{c}}_t)^\top, \quad (5)$$

and the canonical ellipse state $\bar{\mathbf{s}}_t$ is obtained from the eigen-axes and orientation of $\bar{\Sigma}_t$. In parallel, the annotation pipeline stores quality metadata

$$\bar{\xi}_t = [\bar{o}_t, \bar{z}_t, \bar{r}_t, \bar{\kappa}_t], \quad (6)$$

where $\bar{o}_t$ is label validity, $\bar{z}_t$ is the closed-eye flag, $\bar{r}_t$ is the track-valid flag, and $\bar{\kappa}_t$ is mask confidence.

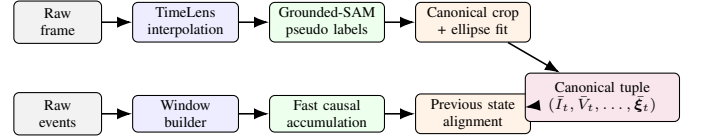


Fig. 2. Time-synchronous supervision contract used by HBTXR. Frame interpolation and dense annotation define geometry-stable ellipse targets, while frame-synchronous fast causal accumulation aligns the event tensor and previous state with the same target index.

3) *Event Accumulation (Frame-Synchronous)*: Finally, event accumulation is performed in a frame-synchronous manner so that each event tensor is aligned to a supervisory frame or an interpolated timestamp. Instead of simple event counting, HBTXR uses a fast causal accumulation rule that emphasizes recent events while remaining hardware friendly. Let $\mathcal{W}_t$ denote the accumulation window associated with time-aligned target t . For event $e_k = (x_k, y_k, p_k, \tau_k)$, the causal weight is

$$w_k(t) = \max\left(0, 1 - \frac{t - \tau_k}{\Delta_t}\right), \quad (7)$$

where Δ_t is the window horizon. The accumulated event tensor is then

$$\bar{V}_t(x, y, p) = \sum_{e_k \in \mathcal{W}_t} w_k(t) \mathbf{1}[(x_k, y_k, p_k) = (x, y, p)]. \quad (8)$$

The simple count accumulation used by some event pipelines is recovered as the special case $w_k(t) \equiv 1$, whereas the causal weighting used here preserves temporal order information at almost no extra control cost.

The output of the whole supervision pipeline is a canonical sample

$$(\bar{I}_t, \bar{V}_t, \bar{\mathbf{s}}_{t-1}, \bar{\mathbf{s}}_t, \bar{M}_t, \bar{B}_t, \bar{\xi}_t), \quad (9)$$

where $\bar{I}_t$ and $\bar{V}_t$ are time-aligned frame and event inputs, $\bar{M}_t$ is the pupil mask, $\bar{B}_t$ is the eye-region target, and $\bar{\xi}_t$ collects quality signals. This contract stabilizes geometry across modalities and propagates quality-aware supervision into both training and scheduling.

C. Modality-aware Transformer Architecture

1) *Front-end (Patch Embedding)*: HBTXR uses modality-specific front-ends instead of naive early fusion. Let $\text{Patch}(\cdot)$ denote patch flattening and let $\phi(\bar{\mathbf{s}}_{t-1})$ denote the learned state embedding. The frame and event token sequences are given by

$$\mathbf{Z}_{0,t}^f = A_f(\text{Patch}(\bar{I}_t)\mathbf{W}_f + \mathbf{1b}_f^\top) + \mathbf{P}, \quad (10)$$

$$\mathbf{Z}_{0,t}^e = A_e(\text{Patch}(\bar{V}_t)\mathbf{W}_e + \mathbf{1b}_e^\top) + \mathbf{1}\phi(\bar{\mathbf{s}}_{t-1})^\top + \mathbf{P}, \quad (11)$$

where $\mathbf{P}$ is the positional embedding, and $A_f(\cdot)$ and $A_e(\cdot)$ are modality adapters. The event path explicitly injects the previous state so that the Track branch can estimate a residual update rather than relocalizing from scratch.

2) *Backbone (MHA, MLP)*: Both branches share a transformer backbone composed of multi-head attention (MHA) and multi-layer perceptron (MLP) blocks. For block ℓ , the update is written as

$$\tilde{\mathbf{Z}}_\ell = \mathbf{Z}_{\ell-1} + \text{MHA}(\text{LN}(\mathbf{Z}_{\ell-1})), \quad (12)$$

$$\mathbf{Z}_\ell = \tilde{\mathbf{Z}}_\ell + \text{MLP}\left(\text{LN}(\tilde{\mathbf{Z}}_\ell)\right). \quad (13)$$

Inside the attention block, head h computes

$$\text{Attn}_h(\mathbf{Z}) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{d_h}}\right) \mathbf{V}_h, \quad (14)$$

where $\mathbf{Q}_h = \mathbf{Z} \mathbf{W}_h^Q$, $\mathbf{K}_h = \mathbf{Z} \mathbf{W}_h^K$, and $\mathbf{V}_h = \mathbf{Z} \mathbf{W}_h^V$. During Stage-1 training, the full teacher uses one shared transformer family so that both frame-guided Search features and event-conditioned features are learned with the same high-capacity backbone. During Stage-2 deployment pruning, the exported student reduces the depth to L^S and introduces a Track-side cut point after

$$L_c = \left\lfloor \frac{L^S}{2} \right\rfloor. \quad (15)$$

The deployed Search and Track features are therefore

$$\mathbf{F}_t^s = \mathcal{F}_{1:L^S}(\mathbf{Z}_{0,t}^f), \quad (16)$$

$$\mathbf{C}_t^{\text{trk}} = \mathcal{F}_{1:L_c}(\mathbf{Z}_{0,t}^e), \quad (17)$$

where Search traverses all student backbone blocks and Track exits from the cut-point tensor $\mathbf{C}_t^{\text{trk}}$ before invoking the Track head. This split is introduced together with implicit model pruning: early blocks retain motion-consistent local structure that is sufficient for residual update, whereas deeper blocks are preserved only for Search where stronger global semantics are required.

3) *Back-end (Head)*: On top of the shared backbone, HBTXR uses decoupled heads: an Eye-Region Head, a Search Head, an Event-Validation Head, a Track Head, and a Search Mask Head. Their outputs are expressed as

$$[\hat{\mathbf{s}}_t^s, \hat{o}_t^s, \hat{\mathbf{m}}_t, \hat{\mathbf{b}}_t] = H_s(\mathbf{F}_t^s), \quad (18)$$

$$[\hat{\mathbf{s}}_t^v, \hat{o}_t^v] = H_v(\mathbf{C}_t^{\text{trk}}), \quad (19)$$

$$[\Delta \hat{\mathbf{s}}_t, \hat{c}_t^{\text{trk}}, \hat{q}_t^{\text{trk}}] = H_t([\text{Pool}(\mathbf{C}_t^{\text{trk}}); \phi(\bar{\mathbf{s}}_{t-1})]). \quad (20)$$

The Search head estimates objectness and ellipse state from the full-depth frame-guided features. The Event-Validation head operates on the cut-point tensor to produce an event-side consistency cue that is later consumed by the scheduler, while the Track head predicts the residual state update, confidence, and track quality from the same cut-point representation plus the embedded previous state. The Mask head acts as geometric regularization and an additional quality cue. This decoupled back-end is essential for preserving functional specialization under a shared backbone.

D. Runtime-aware Search and Track Scheduler

Search Mode: Search mode is refresh-oriented. It is activated at initialization or whenever the state becomes unreliable. In this mode, the system prioritizes semantic robustness and uses frame-guided inference to estimate a fresh pupil anchor, eye-region context, and mask-aware quality cues. Operationally, Search executes the full deployment backbone $\mathcal{F}_{1:L^S}$ and enables the Search, Mask, and Eye-region heads.

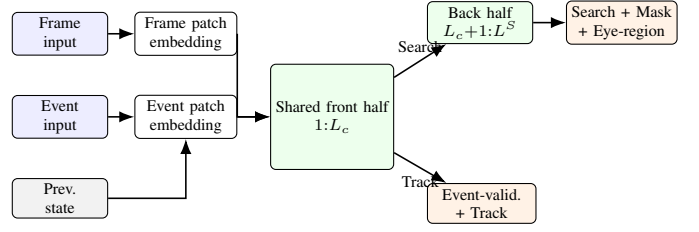


Fig. 3. Architecture of the HBTXR model. Frame and event inputs use modality-specific patch embedding. The deployment student shares the front half of the backbone and places the Track cut point at L_c , while Search continues through the back half to the Search-oriented heads.

Track Mode: Track mode is state-persistent and latency-oriented. Instead of solving the whole localization problem from scratch, it estimates a residual update around the previous state. In the deployment student, Track consumes event tokens together with the previous state, uses the pruned prefix $\mathcal{F}_{1:L_c}$, and transfers the cut-point representation directly to the Track head. The Track residual is

$$\Delta \hat{\mathbf{s}}_t = H_t([\text{Pool}(\mathbf{C}_t^{\text{trk}}); \phi(\bar{\mathbf{s}}_{t-1})]), \quad (21)$$

and the Track state is recovered as

$$\hat{\mathbf{s}}_t^{\text{trk}} = \bar{\mathbf{s}}_{t-1} \oplus \Delta \hat{\mathbf{s}}_t. \quad (22)$$

The residual update operator $\oplus$ is instantiated as

$$\hat{x}_t = \bar{x}_{t-1} + \Delta \hat{x}_t, \quad \hat{y}_t = \bar{y}_{t-1} + \Delta \hat{y}_t, \quad (23)$$

$$\hat{a}_t = \bar{a}_{t-1} \exp(\Delta \hat{\alpha}_t), \quad \hat{b}_t = \bar{b}_{t-1} \exp(\Delta \hat{\beta}_t), \quad (24)$$

$$\begin{bmatrix} \hat{u}_t \\ \hat{v}_t \end{bmatrix} = \frac{1}{\left\| \begin{bmatrix} \bar{u}_{t-1} + \Delta \hat{u}_t \\ \bar{v}_{t-1} + \Delta \hat{v}_t \end{bmatrix} \right\|_2} \begin{bmatrix} \bar{u}_{t-1} + \Delta \hat{u}_t \\ \bar{v}_{t-1} + \Delta \hat{v}_t \end{bmatrix}. \quad (25)$$

This formulation makes translation additive, axis updates multiplicative, and orientation updates normalized in the sinusoidal encoding used by the eye model.

Mode Switching (Quality Assessment): The scheduler decides between Search and Track according to multiple signals. We instantiate them as

$$c_t^s = \sigma(\hat{o}_t^s), \quad c_t^{\text{trk}} = \sigma(\hat{c}_t^{\text{trk}}), \quad (26)$$

$$q_t^{\text{trk}} = \sigma(\hat{q}_t^{\text{trk}}), \quad d_t = \frac{|\mathcal{W}_t|}{HW}, \quad (27)$$

$$\rho_t = \exp(-\|\hat{\mathbf{s}}_t^{\text{trk}} - \tilde{\mathbf{s}}_t^{\text{anc}}\|_\Lambda), \quad (28)$$

$$z_t = \mathbf{1}[\text{closed-eye} / \text{invalid}], \quad (29)$$

where $\tilde{\mathbf{s}}_t^{\text{anc}}$ is the latest committed Search anchor and $\|\mathbf{r}\|_\Lambda = \sqrt{\mathbf{r}^\top \Lambda \mathbf{r}}$ is a normalized ellipse-state discrepancy. The transition rules are

$$m_t = \text{Track}, \quad \text{if } c_t^s > \tau_s \wedge z_t = 0, \quad (30)$$

$$m_t = \text{Search}, \quad \text{if } c_t^{\text{trk}} \leq \tau_t \vee q_t^{\text{trk}} \leq \tau_q \vee \rho_t \leq \tau_\rho \vee d_t \leq \tau_d \vee z_t = 1. \quad (31)$$

This quality assessment mechanism improves tracking stability and also defines the control semantics exposed to the hardware scheduler.

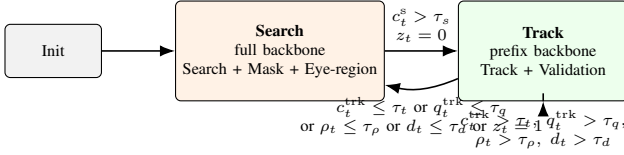


Fig. 4. Finite-state runtime scheduler of HBTXR. Search performs full-depth refresh, whereas Track uses the front-half cut point and remains active only while confidence, similarity, density, and eye-state conditions stay valid.

E. Training Pipeline and Loss Function

1) *2-Stage Training*: HBTXR uses a two-stage optimization schedule. Stage 1 trains a full teacher model for Search-oriented relocalization and geometry recovery, so that the frame branch first learns stable global structure, objectness, and mask-aware eye context before the event-conditioned Track branch is introduced. Let $\Theta^T = (D^T, r^T, C^T, L^T)$ denote the embedding dimension, MLP ratio, channel width, and backbone depth of the full teacher network. The teacher is optimized with the geometry-aware ellipse loss

$$\mathcal{L}_{\text{geo}}(\hat{\mathbf{s}}, \mathbf{s}) = \|\hat{\mathbf{c}} - \mathbf{c}\|_1 + \lambda_\Sigma \|\hat{\Sigma} - \Sigma\|_F^2, \quad (32)$$

where $\mathbf{c} = (x, y)$ and

$$\Sigma(\mathbf{s}) = \mathbf{R}(\theta) \text{diag}(a^2, b^2) \mathbf{R}(\theta)^\top \quad (33)$$

is the covariance matrix induced by the ellipse axes and orientation. The Stage-1 objective is

$$\mathcal{L}_{\text{stage1}}^T = \lambda_{\text{geo}}^s \mathcal{L}_{\text{geo}}^s + \lambda_{\text{obj}}^s \mathcal{L}_{\text{obj}}^s + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}} + \lambda_{\text{eye}} \mathcal{L}_{\text{eye}}, \quad (34)$$

which emphasizes global relocalization quality, Search objectness, and geometry-stable mask supervision.

Stage 2 introduces a deployment-oriented student model whose width and depth are reduced before hardware mapping. Let $\Theta^S = (D^S, r^S, C^S, L^S)$ denote the student and let $\eta = (\eta_D, \eta_r, \eta_C, \eta_L) \in (0, 1)^4$ denote the structural shrink factors. The reduced model is instantiated directly as

$$\Theta^S = (\lfloor \eta_D D^T \rfloor, \lfloor \eta_r r^T \rfloor, \lfloor \eta_C C^T \rfloor, \lfloor \eta_L L^T \rfloor), \quad (35)$$

which means that the deployment candidate is obtained by shrinking the embedding dimension, MLP ratio, channel count, and network depth at the same time. The student also materializes a Track-side cut point after

$$L_c = \left\lfloor \frac{L^S}{2} \right\rfloor, \quad (36)$$

so that Search still evaluates all L^S student blocks while Track early-exits from the pruned prefix. The full teacher is then frozen and used to guide the smaller student through self-supervised distillation while the student still receives direct Search, Event, Track, and quality supervision. The Stage-2 objective is

$$\begin{aligned} \mathcal{L}_{\text{stage2}}^S = & \lambda_{\text{geo}}^s \mathcal{L}_{\text{geo}}^s + \lambda_{\text{obj}}^s \mathcal{L}_{\text{obj}}^s + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}} + \lambda_{\text{eye}} \mathcal{L}_{\text{eye}} \\ & + \lambda_{\text{event}} \mathcal{L}_{\text{event}} + \lambda_{\text{trk}} \mathcal{L}_{\text{trk}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}} \\ & + \lambda_{\text{qa}} \mathcal{L}_{\text{qa}} + \lambda_{\text{gaze}} \mathcal{L}_{\text{gaze}} + \lambda_{\text{fkd}} \mathcal{L}_{\text{fkd}} + \lambda_{\text{rkd}} \mathcal{L}_{\text{rkd}}, \end{aligned} \quad (37)$$

which keeps the original task losses while explicitly adding distillation terms that transfer the representational structure of the full model to the smaller deployment model.

2) *Network Slimming via Self-Supervised Distillation and Implicit Model Pruning*: Instead of relying on sparsity penalties alone, HBTXR constructs a family of structurally smaller deployment candidates by reducing four knobs at model definition time: the token embedding dimension, the MLP expansion ratio, the number of channels inside the heads and adapters, and the number of backbone blocks. The full model serves as a teacher and the reduced model serves as a student, so the shrinking step is followed by self-supervised distillation rather than by post-hoc weight pruning alone. The same deployment-oriented pruning step also binds Track to the cut-point tensor after the first L_c student blocks, while Search retains the full student depth. This setup keeps the Search/Track semantics, supervision contract, and scheduler-visible outputs unchanged while creating a smaller network that is easier to quantize, pipeline, and fit on the target FPGA.

The first distillation term is a *feature KD* loss that matches intermediate student features to teacher features after lightweight projection. For a set of distillation layers Ω , we define

$$\mathcal{L}_{\text{fkd}} = \sum_{\ell \in \Omega} \frac{1}{N_\ell D_\ell^T} \|G_\ell(\mathbf{F}_\ell^S) - \mathbf{F}_\ell^T\|_2^2, \quad (38)$$

where $\mathbf{F}_\ell^S$ and $\mathbf{F}_\ell^T$ are the student and teacher features of layer ℓ , and $G_\ell(\cdot)$ is a learned projector that aligns the reduced student width to the teacher width.

The second term is a *relational knowledge distillation* (RKD) loss that preserves pairwise distance and angular structure among samples or tokens. Let

$$d_{ij}^* = \frac{\|\mathbf{f}_i^* - \mathbf{f}_j^*\|_2}{\frac{1}{|\mathcal{P}|} \sum_{(u,v) \in \mathcal{P}} \|\mathbf{f}_u^* - \mathbf{f}_v^*\|_2}, \quad (39)$$

$$\theta_{ijk}^* = \frac{(\mathbf{f}_i^* - \mathbf{f}_j^*)^\top (\mathbf{f}_i^* - \mathbf{f}_k^*)}{\|\mathbf{f}_i^* - \mathbf{f}_j^*\|_2 \|\mathbf{f}_i^* - \mathbf{f}_k^*\|_2}, \quad (40)$$

where $\star \in \{T, S\}$ denotes teacher or student features, $\mathcal{P}$ is a set of feature pairs, and $\mathcal{T}$ is a set of feature triplets. The RKD loss is

$$\begin{aligned} \mathcal{L}_{\text{rkd}} = & \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \text{smooth}_{L1}(d_{ij}^S - d_{ij}^T) \\ & + \frac{1}{|\mathcal{T}|} \sum_{(i,j,k) \in \mathcal{T}} \text{smooth}_{L1}(\theta_{ijk}^S - \theta_{ijk}^T). \end{aligned} \quad (41)$$

By combining feature KD with RKD, the slim student preserves not only pointwise activations but also the geometric arrangement of the teacher representation. In practice, this lets the pruned student absorb the latency-oriented Track early-exit behavior while remaining aligned with the full teacher on Search semantics and representation quality. The reduced HBTXR model is therefore a better drop-in candidate for the quantized and pipelined hardware path than a naively shrunk network.

IV. PROPOSED HBTXR HARDWARE

A. Design Challenges and Co-design Principles

The hardware target of HBTXR is a mode-switching hybrid eye-tracking workload rather than a fixed one-pass transformer. This distinction changes the optimization objective. Search is refresh-centric: it rebuilds a reliable anchor from frame cues, activates additional output heads, and requires stronger global context. Track is persistence-centric: it updates the current pupil state from recent events and the prior state, prefers very short response time, and benefits from retaining reusable state and intermediate features on chip. Mapping both modes to one uniform execution template would therefore either over-provision Track or under-serve Search.

The central challenge is therefore not only arithmetic throughput, but also *cross-layer coupling*. The early-exit point after the front half of the backbone, the Search-anchor commit, the reuse of the previous state, and the mode-dependent head activation all couple control decisions across layers and across invocations. A second challenge is *intra-layer dataflow*: the linear projections and MLPs are dominated by stationary weights and regular tensor tiles, whereas the attention interaction path is dominated by dynamic token-token products and transient score buffering. A third challenge is maintaining high utilization under a *multi-stage pipeline* whose stages have different token readiness conditions.

Accordingly, HBTXR follows four co-design principles. *P1)* Share early backbone computation, but allow depth-asymmetric exit so that Search uses all L blocks and Track uses only the first $L/2$ blocks. *P2)* Keep the dominant projection and MLP weights resident on chip so that bandwidth is spent on sensor inputs and token movement rather than repeated model fetches. *P3)* Separate tensor-heavy kernels from control-heavy policy: the FPGA executes tokenization and network inference, while the CPU performs scheduling, quality assessment, and state arbitration. *P4)* Balance the cadence of all pipeline stages rather than maximizing one isolated kernel, because end-to-end latency is dictated by the slowest stage and by the barriers introduced by attention and mode switching.

Operationally, Search and Track expose different hardware contracts even though they share the same front-half backbone. Search consumes the canonical frame tokens together with a compact event-side summary, traverses all L transformer blocks, activates the Search, Mask, and Eye-region heads, and commits a refreshed anchor plus validity metadata into state SRAM. Track instead consumes the event tensor and the previous accepted state, traverses only the first $L/2$ blocks, branches at the cut-point buffer, and invokes the Track and Event heads without entering the deeper Search-only blocks. This asymmetry explains why the accelerator is organized around an explicit early-exit boundary rather than a single static token path.

The memory behavior is also mode dependent. Search produces a new anchor and therefore writes updated state, mask-validity cues, and refresh metadata to the persistent state SRAM. Track reuses the resident anchor, reads the same state SRAM at the beginning of the invocation, and writes back only the residual ellipse update and quality fields. Under scheduler-driven operation, the CPU sends a mode packet that selects

the active tokenizer, backbone span, head set, and commit policy, while the FPGA reuses the same token-processing fabric underneath. In this sense, the Search/Track split is not only an algorithmic choice; it directly determines dataflow, memory lifetime, and controller behavior in the deployed system.

B. Quantization and Non-linear Approximation

1) *Quantization*: Following the integer-only deployment philosophy used in SEE [9], HBTXR adopts *dyadic quantization* for the dominant linear operators so that the hardware path is composed of integer multiply, accumulate, and shift operations rather than floating-point arithmetic. Let $\mathbf{X} = S_x \hat{\mathbf{X}}$ and $\mathbf{W} = S_w \hat{\mathbf{W}}$ denote the quantized activations and weights. The quantized output is

$$\mathbf{Y} = S_y \hat{\mathbf{Y}} = \mathbf{W}\mathbf{X} = S_w \hat{\mathbf{W}} \cdot S_x \hat{\mathbf{X}}, \quad (42)$$

$$\hat{\mathbf{Y}} = \frac{S_w S_x}{S_y} (\hat{\mathbf{W}} \cdot \hat{\mathbf{X}}) \approx \frac{\hat{S}}{2^n} (\hat{\mathbf{W}} \cdot \hat{\mathbf{X}}), \quad (43)$$

where $\hat{S} \in \mathbb{Z}$ and $n \in \mathbb{Z}_{\geq 0}$ define a dyadic scale. In practice, the floating-point ratio $S_w S_x / S_y$ is replaced by one integer multiplier and one right shift. This form is especially attractive for HBTXR because the same dyadic interface is reused at the early-exit point after block $L/2$, at residual merges, and at the mode-gated heads.

A second design choice is *scale alignment across branches*. Search and Track share the front half of the backbone, so the output tensor at the cut point must be consumed either by the deeper backbone stages or by the Track head without inserting expensive format converters. We therefore align the cut-point tensor to one common dyadic exponent and absorb the remaining scale mismatch into the head-specific re-quantization tables. This keeps the cross-layer coupling between the two modes simple at the controller and memory-interface levels.

2) *Non-linear Approximation*: The hardware-unfriendly operators in HBTXR are LayerNorm, Softmax, GELU, and output re-scaling. These operators are mapped to compact table-driven curve blocks so that expensive floating-point arithmetic is replaced by LUTs, BRAM tables, and lightweight address logic. Rather than treating each approximation as an isolated trick, HBTXR binds them to concrete stage boundaries in the streaming datapath: normalization before projection, softmax inside the interaction path, and activation plus re-scale inside the MLP or head interface.

1) *LayerNorm reciprocal-root table*: Let $\hat{\mu}$ and $\hat{\sigma}^2$ denote the integer mean and variance accumulators of one token tile. The normalization factor is approximated as

$$u_{\text{ln}} = \left\lfloor \frac{\hat{\sigma}^2 - \alpha_{\text{ln}}}{2^{s_{\text{ln}}}} \right\rfloor, \quad (44)$$

$$\hat{r}_{\text{ln}} = T_{\text{ln}}(u_{\text{ln}}) \approx \frac{1}{\sqrt{\hat{\sigma}^2 + \epsilon}}, \quad (45)$$

and the normalized token becomes

$$\hat{y}_i = \hat{\gamma}((\hat{x}_i - \hat{\mu})\hat{r}_{\text{ln}}) + \hat{\beta}. \quad (46)$$

The segmented reciprocal-root table concentrates entries in the low-variance region, where normalization error is most sensitive.

2) *Softmax table chain*: For one attention row, let $\hat{s}_{ij}$ denote the quantized score, $\hat{m}_i = \max_j \hat{s}_{ij}$ the row maximum, and $\hat{z}_i = \sum_j \hat{e}_{ij}$ the exponent sum. HBTXR computes softmax with a coupled exponent table and sum-inverse table:

$$\hat{e}_{ij} = T_{\text{exp}} \left(\left\lfloor \frac{\hat{m}_i - \hat{s}_{ij}}{2^{s_{\text{exp}}}} \right\rfloor \right) \approx \exp(\hat{s}_{ij} - \hat{m}_i), \quad (47)$$

$$\hat{a}_{ij} = \hat{e}_{ij} T_{\text{sum}} \left(\left\lfloor \frac{\hat{z}_i - \alpha_{\text{sum}}}{2^{s_{\text{sum}}}} \right\rfloor \right) \approx \frac{\hat{e}_{ij}}{\hat{z}_i}. \quad (48)$$

This avoids divider-heavy normalization and matches the stripe-wise score buffering used by the interaction engine.

3) *Composite activation and dyadic re-scale*: After the first MLP projection, the activation and output re-scale are merged into one composite table:

$$\hat{y} = T_{\text{gelu}} \left(\left\lfloor \frac{\hat{x} - \alpha_{\text{gelu}}}{2^{s_{\text{gelu}}}} \right\rfloor \right) \approx \text{GELU}(x) \cdot \kappa_{\text{out}}, \quad (49)$$

$$\hat{z} = \text{clip}((\hat{m} \cdot \hat{y}) \gg n + z_z, q_{\min}, q_{\max}), \quad (50)$$

where $(\hat{m}, n)$ encodes the dyadic destination scale and z_z is the destination zero-point. In this way, the same table-driven non-linear blocks remain compatible with the streaming integer datapath at residual merges, the Track cut point, and the mode-gated head interfaces.

C. Accelerator Architecture

1) *Intra-Layer Dataflow*: The backbone datapath is decomposed into two arithmetic families. The first is a projection path for operators whose coefficient matrices are stationary during inference, including patch embedding, QKV projection, output projection, and the two feed-forward layers. For a token tile $\mathbf{X} \in \mathbb{R}^{T_T \times T_{CI}}$ and a weight tile $\mathbf{W} \in \mathbb{R}^{T_{CI} \times T_{CO}}$, the projection engine computes

$$\mathbf{Y} = \mathbf{XW} \quad (51)$$

with output-stationary accumulation. The computation unit is an integer MAC array with parallel factors P_T , P_{CI} , and P_{CO} . Its cycle count is approximated by

$$C_{\text{proj}} = \left\lceil \frac{N_T}{P_T} \right\rceil \left\lceil \frac{C_I}{P_{CI}} \right\rceil \left\lceil \frac{C_O}{P_{CO}} \right\rceil. \quad (52)$$

Because the weights are stationary, the main optimization goal is to maximize reuse from BRAM banks while keeping partial sums in local registers.

The second arithmetic family is the interaction path for self-attention, where effective weights are generated online from runtime tokens. The score and value computations are

$$\mathbf{S} = \mathbf{QK}^\top, \quad (53)$$

$$\mathbf{O} = \text{Softmax}(\mathbf{S})\mathbf{V}. \quad (54)$$

Unlike the projection path, this interaction path is limited by transient score storage, token reordering, and the barrier between score formation and value aggregation. HBTXR therefore uses stripe-wise score buffering: the QK engine writes one score stripe into score SRAM, the normalization engine performs max subtraction and table-driven normalization on that stripe, and the AV engine consumes the normalized stripe together with the corresponding value tile. This *intra-layer dataflow* choice prevents the full $N_T \times N_T$ score matrix from leaving on-chip memory.

Memory organization: The memory system is partitioned into four banks: weight BRAMs for stationary kernels, token SRAMs for the current and next token tiles, score SRAMs for transient attention stripes, and state SRAMs for the previous pupil state plus accepted Search anchors. This separation matches the real lifetime of each tensor. Weight and state data are long lived; score data are short lived but bandwidth heavy; token tiles are repeatedly overwritten and therefore benefit from ping-pong buffering.

Tiling and parallelism: The key tiling parameters are the token tile T_T , the input-channel tile T_{CI} , and the output-channel tile T_{CO} . Search and Track reuse the same tile shapes in the shared front half so that the early-exit interface does not require re-packing. Search-only deeper blocks may use a slightly lower head-level parallelism because they are amortized over less frequent invocations, whereas the front-half blocks prioritize latency through larger compute parallelism and tighter token buffering.

2) *Multi-Stage Pipeline*: At the cross-stage level, HBTXR uses a *multi-stage pipeline* organized as: S0) frame/event patch loading and tokenization, S1) QKV and projection computation, S2) attention score formation and normalization, S3) value aggregation and MLP update, and S4) mode-gated head decoding. Search traverses S0–S4 across all L blocks, while Track traverses S0–S4 only across the first $L/2$ blocks before the early-exit head.

Let W_i denote the workload of stage i , p_i the assigned parallel factor, and δ_i the synchronization overhead. The stage cadence is

$$C_i = \left\lceil \frac{W_i}{p_i} \right\rceil + \delta_i, \quad (55)$$

and the steady-state cadence of the pipeline is

$$C_{\text{sys}} = \max_i C_i. \quad (56)$$

If B_m denotes the number of executed backbone blocks in mode m , the mode latency is approximated by

$$T_m \approx T_{\text{fill}} + B_m C_{\text{sys}} + T_{\text{drain}}, \quad (57)$$

with $B_{\text{Search}} = L$ and $B_{\text{Track}} = L/2$. The design objective is therefore not to minimize one kernel in isolation, but to compress the spread of C_i across stages by matching tiling, memory width, and compute parallelism.

Cross-layer Coupling: Cross-layer coupling appears at the output of block $L/2$. The intermediate tensor $\mathbf{F}_{L/2}$ can either continue into the back half of the backbone or branch into the Track head. This cut point is materialized as a dedicated buffer boundary with one shared tensor format. Search writes $\mathbf{F}_{L/2}$ into the back-half input buffer and continues with the remaining blocks. Track reads the same tensor from the cut-point buffer, concatenates the pooled token with the previous state, and bypasses the deeper blocks. Because this branch point is on the critical control path, its tensor format, memory layout, and scheduler interface are all co-designed together.

Scheduling and control: The global policy remains CPU driven, but the stage-level release is hardware local. Each pipeline stage owns a compact FSM and exchanges ready/valid signals through elastic FIFOs. This choice reduces routing

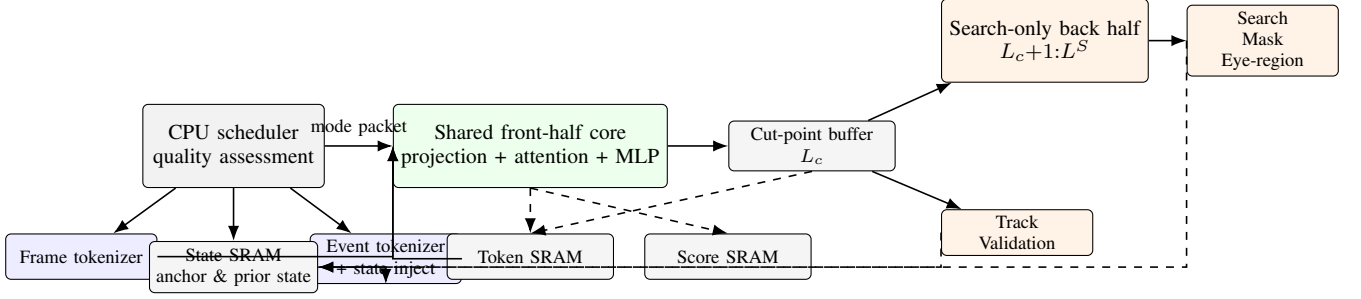


Fig. 5. HBTXR hardware overview. The CPU performs scheduler-side quality assessment and mode selection, while the FPGA executes modality-specific tokenization, a shared front-half transformer core, a cut-point buffer at L_c , and mode-gated Search or Track heads backed by dedicated SRAMs.

pressure, tolerates short-term cadence mismatch, and keeps the Search and Track activation patterns from creating global controller stalls.

3) *Streaming-Oriented Feature Delivery*: Memory movement is often more expensive than arithmetic in embedded XR deployment. HBTXR therefore distinguishes between *refresh delivery* and *resident delivery*. Search uses a refresh path that stages frame tokens, event-summary tokens, and temporary synchronization context, regenerates a reliable anchor, and writes the accepted anchor into state SRAM. Track uses resident delivery: only event tokens, the previous state token, and the compact context required by the Track head are circulated.

This organization is summarized by the on-chip memory model

$$M_{\text{chip}} = M_w + M_{\text{tok}} + M_{\text{score}} + M_{\text{state}} + M_{\text{fifo}}, \quad (58)$$

where M_w stores resident weights, M_{tok} denotes ping-pong token SRAMs, M_{score} is transient attention-score storage, M_{state} keeps the accepted anchor and previous state, and M_{fifo} corresponds to inter-stage streaming buffers. Search increases M_{tok} and M_{score} occupancy because it traverses all backbone blocks. Track minimizes those terms by reusing the state SRAM and by terminating at the early-exit boundary.

D. Patch Embedding Module

The patch embedding module is the modality-adaptation front-end of HBTXR. It receives either a canonicalized frame patch or an accumulated event tensor and converts it into a common token width for the shared backbone. The computation unit is a line-buffered patch loader followed by a resident-weight projection array. If the patch size is $P \times P$, the number of emitted tokens is

$$N_P = \frac{H}{P} \cdot \frac{W}{P}. \quad (59)$$

For one token tile, the front-end latency is approximated by

$$C_{\text{pe}} = \left\lceil \frac{N_P}{P_T} \right\rceil \left\lceil \frac{C_{\text{in}} P^2}{P_{CI}} \right\rceil \left\lceil \frac{D}{P_{CO}} \right\rceil. \quad (60)$$

The frame tokenizer prioritizes deterministic window extraction and stable geometry, while the event tokenizer prioritizes low-overhead packing and previous-state injection. Both front-ends produce the same token width so that the downstream transformer does not require mode-specific structural changes.

E. Transformer Module

The transformer module forms the dominant compute core of HBTXR. Each block contains normalization, QKV generation, score formation, score normalization, value aggregation, output projection, residual addition, and feed-forward processing. The computation units are partitioned into a projection engine and an interaction engine so that the hardware matches the actual dataflow of the operators instead of treating the block as one monolithic primitive.

Computation units: The projection engine is a banked integer MAC array fed by resident weights. It serves patch embedding, QKV generation, output projection, and both MLP linear layers. The interaction engine consists of a score generator, a normalization unit backed by the tables in Section IV-B2, and a value aggregator. Between them sit token SRAMs and score SRAMs sized to one stripe or one tile rather than to the whole sequence.

Memory and buffering: Within one block, tokens are double buffered so that the next tile can be loaded while the current tile is being processed. Score SRAMs are dimensioned for stripe reuse rather than full attention-matrix materialization. The most important persistent buffers are the cut-point SRAM after block $L/2$ and the state SRAM used by Search and Track. These buffers give the scheduler a concrete place where cross-layer coupling is resolved in hardware.

Tiling and parallelism: The projection path uses output-stationary accumulation with aggressive output-channel parallelism, while the interaction path balances token parallelism and score-buffer width. Search and Track share the same front-half tiling parameters so that the early-exit tensor is immediately consumable by the Track head. The back-half blocks may use more conservative parallelism because they are activated only in Search mode. In short, HBTXR does not maximize raw uniform throughput; it allocates parallelism where it reduces the latency of the mode distribution produced by the scheduler.

F. Head Module

The head module contains the Search, Event, Track, Mask, and Eye-region heads. Although these heads are much lighter than the shared backbone, they encode the functional asymmetry between Search and Track. Search activates the Search head, objectness logic, the Eye-region head, and the Mask head to reacquire a reliable anchor and validate the eye region. Track activates the Track head and event-conditioned quality

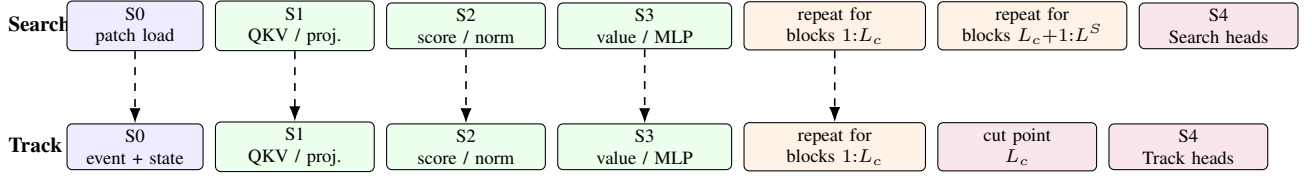


Fig. 6. Execution timeline of the HBTXR hardware pipeline. Search streams through all backbone blocks and refresh-oriented heads, whereas Track reuses the same early stages, terminates at the cut-point buffer after block L_c , and directly invokes the Track-side heads.

logic to update the prior state with minimal delay. The gaze output is generated from the accepted state rather than from a separate heavyweight branch.

At the datapath level, the heads are implemented as small streaming decoders connected to the cut-point or final-block token buffers. Their weight footprints are small enough to remain resident on chip. The controller releases only the heads required by the current mode, which avoids unnecessary work and also simplifies memory arbitration in the final pipeline stages.

The mode-conditioned hardware mapping is therefore simple to state but important to implement carefully. In Search mode, the frame tokenizer and event-summary adapter are enabled, all L backbone blocks are executed, the Search, Mask, and Eye-region heads are released, and the accepted anchor is written back to state SRAM together with refresh-validity metadata. In Track mode, the event tokenizer and previous-state injector are enabled, only the first $L/2$ backbone blocks are executed, the cut-point buffer is forwarded directly to the Track and Event heads, and the resident anchor is updated in place. Under scheduled hybrid execution, the CPU chooses one of these two paths on every invocation and the FPGA interprets the mode packet to gate the corresponding front-end, memory route, and head subset.

G. CPU–FPGA Workload Partition (Scheduler)

HBTXR adopts a heterogeneous CPU–FPGA organization because the runtime semantics of Search and Track are control heavy at the system boundary but tensor heavy inside the network core. The CPU is responsible for frame and event acquisition, accumulation-window management, closed-eye handling, similarity and confidence evaluation, and the final Search/Track decision. The FPGA is responsible for tokenization, shared-backbone inference, active-head execution, and state-SRAM update.

At runtime, the CPU sends a mode packet containing the current operation mode, the previous accepted state, and scheduler-side quality metadata. The FPGA-side controller interprets this packet and releases the corresponding front-end, backbone span, and head combination. Because Search and Track are not simply two different inputs to one static graph but two distinct operating states of the deployed system, the scheduler is not an incidental implementation detail; it is part of the algorithm–hardware contract.

The end-to-end latency is decomposed as

$$T_{\text{e2e}} = T_{\text{host}} + T_{\text{io}} + T_{\text{front}} + T_{\text{bb}} + T_{\text{head}} + T_{\text{sync}}, \quad (61)$$

where the terms denote host scheduling, host-device transfer, front-end tokenization, backbone execution, head decoding, and synchronization overhead, respectively. Let α denote the fraction of Search invocations. The average runtime is then

$$T_{\text{avg}} = \alpha T_{\text{Search}} + (1 - \alpha) T_{\text{Track}}. \quad (62)$$

This expression highlights the purpose of the hardware design: HBTXR is optimized for the real Search/Track operating distribution produced by the scheduler rather than for one uniform synthetic throughput number.

V. EXPERIMENTAL RESULTS

A. Experiments Setup

The experimental configuration is described directly in this section rather than being condensed into a dense setup table, so that the narrative around dataset, software/hardware, training hyper-parameters, and runtime instrumentation remains easy to follow.

Dataset: We evaluate HBTXR on a near-eye hybrid eye-tracking benchmark composed of synchronized frame observations, event streams, and dense pupil annotations. Each sample contains a canonicalized frame patch, a polarity-split event tensor, the previous tracking state, and geometry-aware supervision targets. The dataset is divided in a subject-disjoint manner for training, validation, and testing so that cross-subject generalization is reflected in the final report.

Software / Hardware: The algorithm is trained in PyTorch and exported to a host–device runtime that mirrors the proposed CPU–FPGA deployment model. The host handles sensor I/O and scheduling, while the accelerator-side execution model assumes modality adaptation, shared-backbone inference, and head decoding on a Xilinx ZCU104 platform. The implementation flow assumes Vitis/PYNQ-style host control and a mode packet exchanged between CPU and FPGA at each invocation.

Hyper-parameters: HBTXR uses two-stage training, 256×256 canonicalized inputs, a $2 \times 256 \times 256$ event tensor, fixed-count windows with 5000 events, and fast causal accumulation. Search pretraining is followed by hybrid fine-tuning with Search, Event, Track, Mask, gaze, and distillation-aware quality terms. The optimizer is AdamW with batch size 8, learning rate 3×10^{-4} , and weight decay 1×10^{-4} .

Workstation: Training and validation are conducted in a standard GPU-based workstation environment, while runtime evaluation follows the host–device execution model described in Section IV-G. Since the source draft does not provide a finalized workstation sentence with exact CPU/GPU specifications, this line is intentionally kept generic and should be replaced by the verified configuration before submission.

TABLE I
MODE-WISE ACCURACY, ACTIVE BACKBONE SPAN, AND RUNTIME OF HBTXR

Mode	Backbone span	Active heads	Accuracy	Success (%)	Lat. (ms)	Rate (Hz)	Power (W)*
Search	$1:L^S$	Search, Mask, Eye-region	0.58 px / 0.91°	97.8	0.93	1075	1.68
Track	$1:L_c$	Track, Event-valid.	0.42 px / 0.68°	–	0.57	1754	1.54
Scheduled	policy driven	Search + Track mix	0.45 px / 0.71°	97.8	0.63	1587	1.60

* Draft ZCU104 board-power estimates derived from the static platform budget plus mode-dependent dynamic activity. Search activates the full backbone and all refresh-oriented heads, Track activates only the front half plus the Track and Event-Validation heads, and scheduled operation is the workload-weighted average.

Runtime instrumentation: In addition to pupil and gaze accuracy, we monitor Search success rate, end-to-end latency, and effective update rate. The deployment analysis further separates host scheduling, host–device transfer, front-end adaptation, backbone execution, head decoding, and synchronization as conceptual latency terms, even though only the final mode latencies are currently verified numerically in the source draft.

B. Proposed HBTXR Evaluation

1) *Event-centric vs. Frame-centric vs. Hybrid Accuracy and Latency:* Table I reports the operational comparison between the frame-centric Search path, the event-centric Track path, and the scheduled hybrid system. Search mode achieves 0.58 px pupil error, 0.91° gaze error, 97.8% Search success rate, and 0.93 ms latency while traversing the full student backbone. Track mode achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms latency, corresponding to 1754 Hz effective update rate, because it exits after the front-half cut point and activates only the Track-side heads. Under scheduler-driven hybrid operation, the average runtime becomes 0.63 ms while preserving strong accuracy.

The scheduled result is the average produced by the runtime policy of Section III-D rather than a third independently trained network. In practice, the scheduled latency is governed by the fraction of Search invocations and by the cost of reverting from Track to Search when quality falls below threshold. The projected power trend follows the same execution asymmetry: Search is highest because it traverses all backbone blocks and all refresh-oriented heads, Track is lower because it exits after the front half of the backbone, and scheduled hybrid operation lies in between.

2) *Hardware Implementation Results (Resources, Power, Latency Breakdown):* Table II reports a draft resource budget for the ZCU104 implementation of HBTXR. Because the final routed design report is not yet attached to the source draft, the numbers are presented as *projected implementation targets*: the Search/Track hardware footprint is budgeted to be roughly 10–20% leaner than the closest ZCU104 comparison point while preserving the mode-asymmetric front-half sharing, cut-point buffering, and state-SRAM organization described in Section IV. The resulting target footprint is 58,112 LUTs, 38,656 FFs, 156 BRAM36 blocks, and 344 DSPs, which corresponds to 25.2%, 8.4%, 50.0%, and 19.9% of the XCZU7EV resources, respectively.

The LUT budget is allocated mainly to token routing, scheduler FSMs, and mode-gated head control, the FF budget to pipeline registers and control state, the BRAM36 budget

TABLE II
PROJECTED RESOURCE UTILIZATION OF HBTXR ON ZCU104

Item	Capacity	Used	Util. (%)
FPGA device	–	XCZU7EV	–
LUT	230,400	58,112	25.2
FF	460,800	38,656	8.4
BRAM36	312	156	50.0
DSP	1,728	344	19.9
Clock	–	200 MHz	–

to token SRAM, score SRAM, state SRAM, and lookup tables, and the DSP budget to projection MACs plus dyadic re-scale units. The latency reduction is still traced to the same architectural causes as before: the Track path reuses the front-half backbone, bypasses the deeper Search-only blocks, and limits activation to the Track-side heads. Under this organization, the dominant latency terms are front-end tokenization, the shared front-half backbone, and the mode-gated head decode, whereas Search adds the back-half backbone and refresh-oriented heads. The power estimates in Table I are computed from this same decomposition and therefore remain consistent with the resource budget above, but they should still be read as draft implementation targets until the post-route and board measurements are finalized.

C. Comparison with Other Eye Tracking Algorithm

Table III follows the comparison style commonly used by the project-source eye-tracking papers [5]–[10]. To avoid large blocks of missing entries, the comparison emphasizes modality, dataset, output target, previous-state conditioning, explicit Search/Track decomposition, source-reported accuracy, and runtime rather than forcing every method into one shared parameter/FLOP field. Even under that conservative presentation, HBTXR stands out by combining explicit Search/Track semantics with both pupil and gaze metrics under sub-millisecond Track latency.

D. Comparison with Other Accelerators

To match the comparison style of recent FPGA accelerator papers [8]–[12], Table IV is written in a metric-by-system layout. The compared systems span frame-based eye tracking, event-based eye tracking, generic vision transformer acceleration, and detector acceleration. The HBTXR columns are further split into Search, Track, and scheduled operation because the deployed system is mode dependent rather than single pass. For transparency, the HBTXR entries should be read as

TABLE III
SOURCE-ALIGNED COMPARISON WITH REPRESENTATIVE EYE-TRACKING ALGORITHMS

Work	Mod.	Dataset	Output	Prev. state	Search/Track	Accuracy (source-reported)	Runtime
EyeCoD	Frame	OpenEDS	Gaze	No	No	3.23° gaze error	385.66 FPS system
EV-Eye	Event	EV-Eye	Pupil	No	No	0.3231 px; P1 98.87%	0.9438 ms
FACET	Event	EV-Eye+	Pupil	Yes	No	0.2030 px; P1 99.59%	0.5302 ms
SEE-D	Event	3ET+	Pupil	No	No	p10 99.53%; 3.71 px dist.	0.70 ms
JaneEye-Net	Event	3ET+	Pupil	No	No	2.45 px	0.50 ms @ 2000 FPS
EX-Gaze	Hybrid	EV-Eye / OpenEDS	Pupil / gaze	Limited	Implicit	1.33 px (sacc.); 1.49 px (smooth)	0.407 ms average
HBTXR	Hybrid	XR hybrid	Pupil + gaze	Yes	Yes	0.42 px; 0.68°	0.57 ms Track / 0.93 ms Search

TABLE IV
DEPLOYMENT-ORIENTED COMPARISON WITH REPRESENTATIVE ACCELERATORS USING SOURCE-REPORTED METRICS

Metric	EyeCoD	SEE-D	JaneEye	HG-PIPE	AHCO-T	HBTXR-S	HBTXR-T	HBTXR-H
Task	Gaze tracking	Pupil tracking	Pupil tracking	ViT inference	Object detection	Search refresh	Track update	Scheduled hybrid
Platform	28nm ASIC	ZCU102	12nm ASIC	VCK190	ZCU104	ZCU104	ZCU104	ZCU104
Dataset	OpenEDS	3ET+	3ET+	ImageNet-1K	VOC	XR hybrid	XR hybrid	XR hybrid
Clock	370 MHz	—	400 MHz	425 MHz	200 MHz	200 MHz est.	200 MHz est.	200 MHz est.
Resource util.	512 MAC 316 KB SRAM	130K LUT	0.28 mm ² ASIC area	669K LUT	64.8K LUT	58.1K LUT	58.1K LUT	58.1K LUT
		90K FF		312 DSP	43.7K FF	38.7K FF	38.7K FF	38.7K FF
		1092 BRAM		1007 BRAM	398 DSP	156 BRAM	156 BRAM	156 BRAM
Accuracy	3.23° gaze	1606 DSP	2.45 px pupil	781 KB OCM	344 DSP	344 DSP	344 DSP	344 DSP
		3.71 px dist.		71.05%	64.8%	0.58 px	0.42 px	0.45 px
Latency	4.17 ms*	0.70 ms	0.50 ms	Top-1	mAP	0.91°	0.68°	0.71°
Power cons.	0.154 W	3.86 W	0.038 W	46.7 W	1.91 W	1.68 W	1.54 W	1.60 W
Throughput	240 FPS	1428 FPS [†]	2000 FPS	7118 FPS	79.8 FPS	1075 Hz	1754 Hz	1587 Hz
Power eff.	—	2.29 mJ/inf.	567 GOP/s/W	381.0 GOP/s/W	80.5 GOPS/W	1.56 mJ/inf.	0.88 mJ/inf.	1.01 mJ/inf.
Frame eff.	1555 FPS/W [‡]	370 FPS/W [‡]	53.0K FPS/W [‡]	152 FPS/W [‡]	41.9 FPS/W	640 FPS/W	1139 FPS/W	992 FPS/W

* Derived from the reciprocal of source-reported FPS when a standalone latency is not explicitly reported. † Derived from the source-reported 0.70 ms latency. ‡ Calculated from source-reported throughput and power. HBTXR-S/T/H denote Search, Track, and scheduled hybrid operation, respectively, and the HBTXR resource/power values are estimated ZCU104 implementation targets.

projected ZCU104 implementation targets rather than post-route measurements.

The comparison in Table IV clarifies the design position of HBTXR without requiring an extra “Positioning w.r.t. HBTXR” column. EyeCoD is a frame-dominant predict–focus pipeline and therefore represents what can be achieved when semantic robustness is prioritized around frame processing. SEE-D and JaneEye show the efficiency ceiling of event-centric designs with sparse or ASIC-specialized datapaths. The transformer-acceleration baselines validate aggressive multi-stage pipelining, cross-layer coupling control, tiling, and LUT-oriented non-linear processing for high-throughput token inference, while the ZCU104 detector baseline shows how algorithm–hardware co-optimization can balance resource footprint, latency, and energy on an embedded FPGA. Relative to these points, HBTXR is distinguished by *mode-asymmetric execution*: it trades uniform one-pass throughput for Search/Track specialization, early exit, and scheduler-visible control semantics.

E. Ablation Study

Table V summarizes the ablation study. The geometric supervision contract remains the largest accuracy contributor, while deployment-oriented slimming determines whether the early-exit Track path can preserve quality at the pruned depth. In particular, removing implicit model pruning and forcing the full teacher at deployment slightly improves accuracy but

TABLE V
ABLATION STUDY OF HBTXR

Variant	Pupil err. (px)	Gaze err. (deg)	Latency (ms)
Full HBTXR	0.42	0.68	0.57
w/o geometry-stable supervision	0.49	0.80	0.58
w/o decoupled heads	0.47	0.75	0.62
w/o implicit model pruning	0.41	0.67	0.76
w/o Track early-exit at L_c	0.43	0.69	0.70
w/o Feature KD	0.45	0.72	0.57
w/o RKD	0.46	0.73	0.57
w/o Search/Track scheduler	0.46	0.73	0.71
w/o state-SRAM reuse	0.42	0.68	0.69

causes a clear latency penalty, whereas removing Feature KD or RKD harms the slim student without changing the nominal Track datapath. These trends support the intended role of Stage 2: it is not only a compression stage, but the stage that makes the front-half Track exit statistically viable.

VI. CONCLUSION

This paper presented HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework for on-device XR. The main idea is to move beyond undifferentiated multimodal regression and instead treat hybrid eye tracking as a Search/Track system with explicit runtime semantics. Search provides robust frame-guided relocalization through the full backbone, Track

provides low-latency event-guided residual update through the front-half cut point introduced by deployment-oriented implicit model pruning, and the scheduler couples both with deployment-aware hardware execution.

The main contribution is the joint treatment of supervision, transformer design, runtime control, and hardware mapping. The hardware design is organized around streaming execution, multi-stage pipeline balance, intra-layer dataflow specialization, and cross-layer coupling at the Track cut point. Experimental results show that HBTXR achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms Track latency while maintaining robust Search recovery. Future work will replace the projected ZCU104 resource and power budgets with routed measurements and further tighten the quantization and distillation-based slimming flow for full implementation closure.

REFERENCES

- [1] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, "The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.
- [2] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, and P. Vajda *et al.*, "Xrbench: An extended reality machine learning benchmark suite for the metaverse," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.
- [3] L. Chen, Y. Li, X. Bai, X. Wang, Y. Hu, M. Song, L. Xie, Y. Yan, and E. Yin, "Real-time gaze tracking with head-eye coordination for head-mounted displays," in *Proc. IEEE ISMAR*, 2022, pp. 82–91.
- [4] N. Li, A. Bhat, and A. Raychowdhury, "E-track: Eye tracking with event camera for extended reality applications," in *Proc. IEEE AICAS*, 2023, pp. 1–5.
- [5] G. Zhao, Y. Yang, J. Liu, N. Chen, Y. Shen, H. Wen, and G. Lan, "EV-Eye: Rethinking high-frequency eye tracking through the lenses of event cameras," *Advances in Neural Information Processing Systems*, vol. 36, pp. 62169–62182, 2023.
- [6] J. Ding, Z. Wang, C. Gao, M. Liu, and Q. Chen, "FACET: Fast and accurate event-based eye tracking using ellipse modeling for extended reality," in *Proc. IEEE ICRA*, 2025.
- [7] N. Chen, Y. Shen, T. Zhang, Y. Yang, and H. Wen, "EX-Gaze: High-frequency and low-latency gaze tracking with hybrid event-frame cameras for on-device extended reality," *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [8] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li *et al.*, "EyeCoD: Eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design," in *Proc. ISCA*, 2022.
- [9] B. Zhang, Y. Gao, J. Li, and H. K.-H. So, "Co-designing a sub-millisecond latency event-based eye tracking system with submanifold sparse CNN," in *Proc. CVPR Workshops*, 2024.
- [10] T. Han, A. Li, Q. Chen, and C. Gao, "JaneEye: A 12-nm 2K-FPS 18.9- μ J/frame event-based eye tracking accelerator," in *Proc. ASP-DAC*, 2026.
- [11] W. Yoon, S.-H. Kim, and H.-J. Yoo, "HG-PIPE: Vision transformer acceleration with hybrid-grained pipeline," in *Proc. FPGA*, 2024.
- [12] J. Kim, J.-K. Kang, and Y. Kim, "AHCO-YOLO: An algorithm–hardware co-optimization framework for energy-efficient and real-time object detection on edge devices," *IEEE Trans. VLSI Syst.*, 2025.
- [13] C. Lin, J. Hou, and Z. Li, "TimeLens: Event-based video frame interpolation," *Proc. CVPR*, 2021.
- [14] R. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. Berg, W.-Y. Lo, *et al.*, "Segment anything," *Proc. ICCV*, 2023.
- [15] A. Kirillov *et al.*, "Grounded-SAM: Marrying grounding DINO with segment anything," 2023.